

CMSC5743 Lab 02: GEMM-2

Runzhe Tao

School of Integrated Circuits

Peking University

`rztao25@stu.pku.edu.cn`

July 9, 2026

Why Do We Need Faster Matrix Multiplication?

- Naive matrix multiplication: $\mathcal{O}(n^3)$ — requires n^3 scalar multiplications.
- On modern hardware, **multiplication is more expensive** than addition (higher latency, more energy).
- **Key idea:** Can we trade extra additions for fewer multiplications?

A Simple Example: 2×2 Matrix Multiply

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

Naive: **8 multiplications**, 4 additions.

Strassen (1969): **7 multiplications**, 18 additions — the first algorithm below $\mathcal{O}(n^3)$!

Strassen Algorithm

Recap on the Strassen Algorithm: to calculate the matrix multiplication $M \times N$, we first split the matrices into:

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \quad N = \begin{pmatrix} E & F \\ G & H \end{pmatrix}.$$

Then, we calculate the intermediate matrices:

$$S_1 = (B - D)(G + H)$$

$$S_2 = (A + D)(E + H)$$

$$S_3 = (A - C)(E + F)$$

$$S_4 = (A + B)H$$

$$S_5 = A(F - H)$$

$$S_6 = D(G - E)$$

$$S_7 = (C + D)E.$$

Final result from S_1 - S_7 : $C = \begin{pmatrix} S_1 + S_2 - S_4 + S_6 & S_4 + S_5 \\ S_6 + S_7 & S_2 - S_3 + S_5 - S_7 \end{pmatrix}$

Strassen Algorithm

Strassen Algorithm in a divide-and-conquer style. Split into submatrix:

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \quad N = \begin{pmatrix} E & F \\ G & H \end{pmatrix}.$$

```
int Divide = (int)(N/2);  
  
int A11[SIZE][SIZE], A12[SIZE][SIZE], A21[SIZE][SIZE], A22[SIZE][SIZE];  
int B11[SIZE][SIZE], B12[SIZE][SIZE], B21[SIZE][SIZE], B22[SIZE][SIZE];
```

```
//dividing the matrices in 4 sub-matrices:  
for (i = 0; i < Divide; i++)  
{  
    for (j = 0; j < Divide; j++)  
    {  
        A11[i][j] = A[i][j];  
        A12[i][j] = A[i][j + Divide];  
        A21[i][j] = A[i + Divide][j];  
        A22[i][j] = A[i + Divide][j + Divide];  
  
        B11[i][j] = B[i][j];  
        B12[i][j] = B[i][j + Divide];  
        B21[i][j] = B[i + Divide][j];  
        B22[i][j] = B[i + Divide][j + Divide];  
    }  
}
```

SIZE is the dimension n of the original $n \times n$ matrix.

Strassen Algorithm

Then we calculate the intermediate matrices:

$$S_1 = (B - D)(G + H)$$

$$S_2 = (A + D)(E + H)$$

$$S_3 = (A - C)(E + F)$$

$$S_4 = (A + B)H$$

$$S_5 = A(F - H)$$

$$S_6 = D(G - E)$$

$$S_7 = (C + D)E.$$

Let's look at an example of $S_2 = (A + D)(E + H)$:

```
MatrixAdd(A11, A22, AResult, Divide); // a11 + a22
MatrixAdd(B11, B22, BResult, Divide); // b11 + b22
StrassenAlgorithm(AResult, BResult, P1, Divide); // p1 = (a11+a22) * (b11+b22)
```

Note that different S_i have different structures. For instance, $S_5 = A(F - H)$ involves **only one** sub-matrix on the left, while S_2 is a product of **two sums**.

In the code, A11, A22, B11, B22 correspond to A, D, E, H, and P1 is our S_2 .

Strassen Algorithm

Note that Strassen is recursive:

Algorithm 3 Strassen's Algorithm

```
function STRASSEN( $M, N$ )
  if  $M$  is  $1 \times 1$  then
    return  $M_{11}N_{11}$ 
  end if
  Let  $M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$  and  $N = \begin{pmatrix} E & F \\ G & H \end{pmatrix}$ 
  Set  $S_1 = \text{STRASSEN}(B - D, G + H)$ 
  Set  $S_2 = \text{STRASSEN}(A + D, E + H)$ 
  Set  $S_3 = \text{STRASSEN}(A - C, E + F)$ 
  Set  $S_4 = \text{STRASSEN}(A + B, H)$ 
  Set  $S_5 = \text{STRASSEN}(A, F - H)$ 
  Set  $S_6 = \text{STRASSEN}(D, G - E)$ 
  Set  $S_7 = \text{STRASSEN}(C + D, E)$ 
  return  $\begin{pmatrix} S_1 + S_2 - S_4 + S_6 & S_4 - S_5 \\ S_6 + S_7 & S_2 - S_3 + S_5 - S_7 \end{pmatrix}$ 
end function
```

```
MatrixAdd(A11, A22, AResult, Divide); // a11 + a22
MatrixAdd(B11, B22, BResult, Divide); // b11 + b22
StrassenAlgorithm(AResult, BResult, P1, Divide); // p1 = (a11+a22) * (b11+b22)
```

Complexity Analysis: Naive vs. Strassen

- Naive: 8 sub-matrix multiplications per level $\Rightarrow \mathcal{O}(n^3)$
- Strassen: 7 sub-matrix multiplications per level $\Rightarrow \mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.807})$

Matrix size n	Naive mults	Strassen mults	Speedup
2	8	7	1.1×
4	64	49	1.3×
8	512	343	1.5×
16	4 096	2 401	1.7×
64	262 144	117 649	2.2×
256	16 777 216	5 764 801	2.9×

Key insight: Saving just 1 multiplication per level is *amplified* by recursion — the gap grows as n increases.

From Matrix Multiplication to Convolution

- **Strassen's idea:** reduce the number of multiplications in matrix multiply via algebraic re-formulation.
- **Same principle applies to convolution:** the Winograd algorithm reduces the number of multiplications in convolution by working in a *transform domain*.

Why Winograd Matters for CNNs

- Convolution layers dominate CNN compute ($> 90\%$ of FLOPs).
- For 3×3 kernels, Winograd $F(2 \times 2, 3 \times 3)$ reduces multiplications by up to $2.25\times$ compared to direct convolution.
- Widely adopted: cuDNN, TVM, NNPACK all implement Winograd-based convolution.

Now we move on to the Winograd Algorithm for convolution. Recap on the 2-D Winograd with 4×4 input feature D and 3×3 kernel K :

$$D = \begin{bmatrix} d_{00} & d_{01} & d_{02} & d_{03} \\ d_{10} & d_{11} & d_{12} & d_{13} \\ d_{20} & d_{21} & d_{22} & d_{23} \\ d_{30} & d_{31} & d_{32} & d_{33} \end{bmatrix}$$

$$K = \begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}$$

(Here D denotes the input data tile and K denotes the convolution kernel, consistent with the formulas below.)

Understanding `im2col`: From Convolution to Matrix Multiply

1D example: input $[d_0, d_1, d_2, d_3]$, kernel $[k_0, k_1, k_2]$, stride 1.

Sliding-window convolution $\rightarrow$ matrix multiplication:

$$\underbrace{\begin{pmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{pmatrix}}_{\text{im2col matrix}} \begin{pmatrix} k_0 \\ k_1 \\ k_2 \end{pmatrix} = \begin{pmatrix} r_0 \\ r_1 \end{pmatrix}$$

Each row of the `im2col` matrix is one **receptive field** (sliding window).

2D generalization: flatten each 3×3 patch into a row $\Rightarrow$ convolution becomes a single matrix multiply.

- **Pro:** leverages highly optimized GEMM routines.
- **Con:** memory overhead from duplicated elements in overlapping windows.

After `im2col` operation, we can split

$$\left[\begin{array}{ccc|ccc|ccc} d_{00} & d_{01} & d_{02} & d_{10} & d_{11} & d_{12} & d_{20} & d_{21} & d_{22} \\ d_{01} & d_{02} & d_{03} & d_{11} & d_{12} & d_{13} & d_{21} & d_{22} & d_{23} \\ d_{10} & d_{11} & d_{12} & d_{20} & d_{21} & d_{22} & d_{30} & d_{31} & d_{32} \\ d_{11} & d_{12} & d_{13} & d_{21} & d_{22} & d_{23} & d_{31} & d_{32} & d_{33} \end{array} \right] \begin{bmatrix} k_{00} \\ k_{01} \\ \frac{k_{02}}{k_{10}} \\ k_{11} \\ \frac{k_{12}}{k_{20}} \\ k_{21} \\ k_{22} \end{bmatrix} = \begin{bmatrix} r_{00} \\ r_{01} \\ r_{10} \\ r_{11} \end{bmatrix}$$

$$\begin{bmatrix} D_{00} & D_{10} & D_{20} \\ D_{10} & D_{20} & D_{30} \end{bmatrix} \begin{bmatrix} \vec{k}_0 \\ \vec{k}_1 \\ \vec{k}_2 \end{bmatrix} = \begin{bmatrix} \vec{r}_0 \\ \vec{r}_1 \end{bmatrix} = \begin{bmatrix} M_0 + M_1 + M_2 \\ M_1 - M_2 - M_3 \end{bmatrix}$$

$$\begin{aligned} M_0 &= (D_{00} - D_{20})\vec{k}_0 \\ M_1 &= (D_{10} + D_{20})\frac{\vec{k}_0 + \vec{k}_1 + \vec{k}_2}{2} \\ M_2 &= (D_{20} - D_{10})\frac{\vec{k}_0 - \vec{k}_1 + \vec{k}_2}{2} \\ M_3 &= (D_{10} - D_{30})\vec{k}_2 \end{aligned}$$

Here k_0, k_1, k_2 are column vectors obtained by reshaping the 3×3 kernel K column-by-column.

The formulation of Winograd convolution is:

$$Y = A^T \left[\underbrace{[GgG^T]}_{\text{kernel transform}} \odot \underbrace{[B^T dB]}_{\text{input transform}} \right] A \quad (1)$$

Step-by-step interpretation:

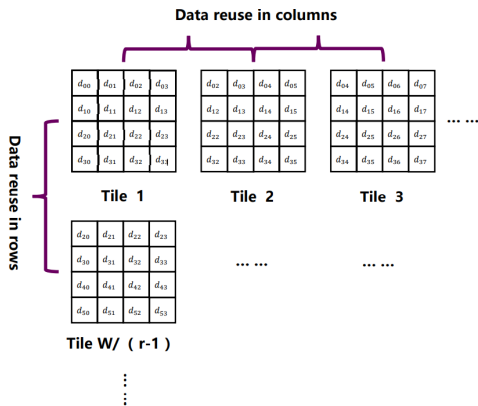
- ① GgG^T : pre-transform the kernel g into the Winograd domain.
- ② $B^T dB$: transform the input tile d into the Winograd domain.
- ③ $\odot$: element-wise multiplication — the **only multiplications** (minimized!).
- ④ $A^T(\cdot)A$: inverse-transform the result back to the spatial domain.

The transforms for $F(2 \times 2, 3 \times 3)$ are:

$$B^T = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad G = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}$$

Winograd: Tiling Strategy

A feature map is usually larger than 4×4 . We handle it by splitting into 4×4 tiles with **overlapping**:



- Each tile is $4 \times 4 \Rightarrow$ reuses the same transforms A, B, G .
- Overlap step = 2 (for 3×3 kernel): each tile produces 2×2 output; adjacent tiles overlap by 2 rows/columns so outputs merge seamlessly.

Winograd: A Concrete Example

Verify $F(2 \times 2, 3 \times 3)$ with a 4×4 input tile d and 3×3 kernel g :

$$d = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}, \quad g = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Direct convolution (stride 1, no padding), e.g. top-left output element:

$$r_{00} = \sum_{i,j} d_{ij} g_{ij} = d_{00}g_{00} + d_{02}g_{02} + d_{11}g_{11} + d_{20}g_{20} + d_{22}g_{22} = 1 + 3 + 6 + 9 + 11 = 30$$

Full result: $Y_{\text{direct}} = \begin{pmatrix} 30 & 35 \\ 50 & 55 \end{pmatrix}$

Exercise: Compute $B^T d B$, $G g G^T$, their element-wise product, and apply $A^T(\cdot)A$.

Verify you get the same $\begin{pmatrix} 30 & 35 \\ 50 & 55 \end{pmatrix}$!

THANK YOU!